GeeksFinder: Job Alert & Resume Tailoring SaaS

Business Plan · PRD · TRD · Execution Plan

Oct 14, 2026 ·

1. Business plan

We will launch a ₹0-cost MVP in 30 days that sends final-year students matched job alerts and tailors their resume per job, then charge ₹10 every 15 days by UPI Autopay.

Problem

Final-year students apply to many companies but waste hours finding relevant jobs, rewriting resumes for each role, and finding whom to contact. Most job portals show generic listings, not jobs matched to a student's actual skills.

Solution

A web app plus Telegram bot that:

- Reads the student's resume and builds a skill profile
- Fetches new jobs daily from multiple sources (including LinkedIn-sourced listings via aggregator APIs)
- Sends matched jobs daily, weekly or monthly on Telegram and email
- Tailors the resume to each job description in one click
- Finds HR contact emails and drafts personalized emails and LinkedIn notes
- Tracks every application from Saved to Offer

Target users

Segment	Need	Where to reach them
Final-year engineering, BCA/MCA, BSc students	First job or internship	College WhatsApp groups, placement cells

Segment	Need	Where to reach them
Recent graduates (0–1 year)	Fresher jobs off-campus	Telegram job channels, LinkedIn, Instagram
Tier-2/3 college students	Off-campus access their college lacks	Local college ambassadors

Pricing

Plan	Price	Includes
Free	₹0	Weekly alerts, 1 resume tailoring, tracker
Pro	₹10 per 15 days, UPI Autopay mandatory	Daily alerts, 10 tailored resumes per cycle, HR email finder, email drafts, follow-up reminders

New users get a 7-day free trial, but must set up the autopay mandate on day 1. The first ₹10 is debited on day 8.

Revenue math

Paying users	Monthly revenue (₹20/user)	After ~2.4% gateway fees
100	₹2,000	~₹1,950
500	₹10,000	~₹9,760
2,000	₹40,000	~₹39,000

Fee figures are approximate; check Razorpay's current subscription pricing.

Cost strategy

Month 1 runs entirely on free tiers: Vercel, Supabase, Gemini/Groq AI, Telegram, Brevo/Resend email, and free job API quotas. Paid upgrades start only once revenue covers them, expected around 300–500 active users.

2. PRD (Product Requirements Document)

The MVP must let a student go from resume upload to receiving matched jobs, a tailored resume and an HR email draft in under 10 minutes.

Goals

- Deliver relevant, fresher-friendly job matches daily
- Cut resume-tailoring time per application from ~30 minutes to under 2 minutes
- Make contacting HR a one-click action
- Convert at least 10% of free users to paid autopay within 60 days

Non-goals (MVP)

- Auto-applying to jobs on the student's behalf
- Scraping LinkedIn or auto-sending LinkedIn messages
- WhatsApp alerts (paid channel; month 2+)
- Mobile apps (responsive web + Telegram is enough)

Persona

Priya, 21, final-year B.Tech (CSE) at a tier-3 college. Knows Python, React and SQL. Her college gets few product-company drives. She checks Telegram many times a day but rarely opens email. She can pay ₹10–20 via UPI but not ₹500.

User stories

1. As a student, I upload my resume PDF and see my extracted skills, education and projects, which I can edit.
2. As a student, I set preferred roles, locations (incl. remote), and job type (full-time / internship).
3. As a student, I choose alert frequency (daily, weekly, monthly) and channel (Telegram, email, both).
4. As a student, I receive a digest of my top matches with a match score and buttons: Apply, Tailor Resume, Email HR, Save.
5. As a student, I tailor my resume to one job, review a before/after diff, and download an ATS-friendly PDF.

6. As a student, I click Email HR, see the found contact, edit the AI draft, and send it from my own Gmail.
7. As a student, I copy a 300-character LinkedIn note for the recruiter.
8. As a student, I track applications across Saved, Applied, Interview, Offer, Rejected.
9. As a student, I get a follow-up reminder 5 days after emailing HR with no reply.
10. As a student, I subscribe with UPI Autopay, see the next debit date, and can cancel anytime.

Functional requirements

Area	Requirement	Priority
Auth	Google login + email OTP	P0
Resume	PDF upload (max 2 MB), AI parse to structured profile, editable	P0
Jobs	Daily ingestion from 3+ sources, dedupe, fresher tagging	P0
Matching	Score 0–100 per user-job; store matches ≥ 60	P0
Alerts	Daily / weekly / monthly via Telegram + email; quiet hours	P0
Tailoring	Rephrase-only AI rewrite, diff view, PDF export; 1 free, 10 per Pro cycle	P0
HR outreach	Email finder chain, AI email draft, Gmail send (or copy + open Gmail), LinkedIn note	P0
Tracker	Kanban of applications, auto-add on Apply/Email	P1
Follow-ups	Telegram reminder 5 days after email	P1
Payments	Razorpay subscription, UPI Autopay mandatory for Pro, 7-day trial	P0
Do-not-contact	HR can be flagged; hidden for all users	P1
Admin	User count, paid count, alerts sent, AI usage	P1

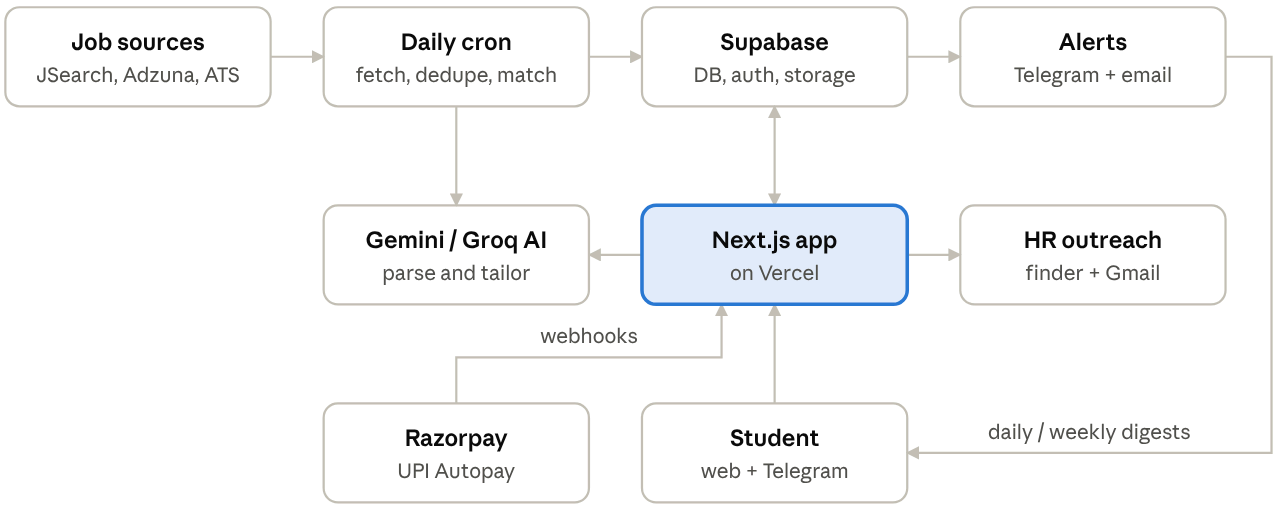
Success metrics

Metric	Target by day 60
Signups	1,000
Activation (resume uploaded + alert set)	60% of signups
Weekly active users	40% of signups
Free → Pro conversion	10%
Autopay renewal success	85% of debits
Tailored resumes per active user per week	2+

3. TRD (Technical Requirements Document)

One Next.js app on Vercel with Supabase as database, auth and storage; scheduled jobs on GitHub Actions do ingestion, matching and alerts, with AI used only for parsing and tailoring.

A daily cron feeds matches; the app handles tailoring, outreach and payments



system architecture · 9 components

Jobs flow left to right into Supabase each morning; everything the student triggers goes through the Next.js app.

Tech stack (free tier)

Layer	Choice	Why
Frontend + API	Next.js (App Router) on Vercel Hobby	One codebase, free hosting
Database, auth, files	Supabase (Postgres, Auth, Storage)	Free tier, row-level security
Scheduled jobs	GitHub Actions cron (Python or Node scripts)	Free minutes, easy logs
AI	Google Gemini Flash (primary), Groq Llama (fallback)	Free tiers, JSON output
Job sources	JSearch (RapidAPI), Adzuna India, Greenhouse/Lever/Ashby public boards, Remotive	Legal, LinkedIn-sourced via aggregator
Telegram	Telegram Bot API (webhook to Next.js route)	Free, instant
Email alerts	Brevo or Resend	Free daily quota
HR email send	Gmail API (user OAuth)	Mail from student's own inbox
Email finder	Job-post extraction, careers-page scraper, Hunter.io / Apollo free credits	Layered, cached
PDF	@react-pdf/renderer, single-column template	ATS-friendly
Payments	Razorpay Subscriptions + UPI Autopay	Recurring ₹10 debits
Monitoring	Vercel logs, Supabase logs, Sentry free	Errors and failures

Check every provider's current free-tier limits before building; they change often.

Data model

Table	Key columns
users	id, name, email, phone, telegram_chat_id, plan (free/pro), created_at

Table	Key columns
profiles	user_id, skills[], education, projects (jsonb), experience_level, preferred_roles[], locations[], job_type, raw_resume_url
alert_settings	user_id, frequency (daily/weekly/monthly), channels[], quiet_hours, last_sent_at
jobs	id, source, source_id, title, company, company_domain, location, remote, description, skills[], fresher_friendly, apply_url, contact_email, posted_at, hash (dedupe)
matches	user_id, job_id, score, reasons (jsonb), notified (bool), created_at
tailored_resumes	id, user_id, job_id, content (jsonb), pdf_url, created_at
company_contacts	company_domain, email, name, role, source, verified, do_not_contact
outreach	id, user_id, job_id, contact_email, channel (email/linkedin), status, sent_at, follow_up_at
applications	user_id, job_id, stage, notes, updated_at
subscriptions	user_id, razorpay_subscription_id, status, current_period_end, trial_end
payments	id, user_id, razorpay_payment_id, amount, status, paid_at
usage	user_id, period_start, tailor_count, finder_count

Enable Supabase row-level security so each user reads only their own rows.

Job ingestion and matching pipeline

1. **Fetch (06:00 IST daily):** pull new jobs from each source for target role keywords and Indian cities + remote.
2. **Dedupe:** hash of lowercase company + title + city; skip existing.
3. **Enrich (AI, once per job):** extract required skills, experience range, fresher_friendly flag, and any contact email in the description, returned as JSON.

4. **Match (plain code, no AI):** score = 50 × skill overlap + 20 × role fit + 15 × location fit + 10 × fresher bonus + 5 × recency. Save matches ⩾ 60 with reasons.
5. **Notify (07:30 IST):** daily users get top 5–10 un-notified matches; weekly digests on Monday; monthly on the 1st. Pro-only daily; free users get weekly.

AI usage

Task	When	Model	Rule
Resume parse	On upload	Gemini Flash	Strict JSON schema output
Job enrich	Once per job	Gemini Flash	Batch 10 jobs per call
Resume tailor	On user click	Gemini (best free model)	Rephrase only; never add skills, companies or numbers
HR email + LinkedIn note	On user click	Gemini Flash / Groq	Under 150 words; note under 300 characters

Cache every AI result. If Gemini hits a rate limit, retry once, then fall back to Groq.

Notifications

- **Telegram:** user clicks "Connect Telegram", gets a deep link `t.me/<bot>?start=<one-time-code>`; the bot's webhook saves telegram_chat_id. Messages use inline buttons linking to the web app.
- **Email:** HTML digest via Brevo/Resend, with unsubscribe link and plan upsell for free users.
- **Rate safety:** stay under ~25 Telegram messages per second; queue sends.

HR email finding chain

On "Email HR", stop at the first hit:

1. Contact email extracted from the job post
2. Cached company_contacts row (not flagged do_not_contact)
3. careers@/hr@/jobs@ found on the company's own website
4. Hunter.io / Apollo lookup (one credit, then cached)
5. Name + domain pattern guess, verified before use

6. Fallback: LinkedIn note only

Gmail sending is capped at 15 emails per user per day. Until Google OAuth verification is approved, use "copy email + open Gmail compose".

Payments: Razorpay Subscriptions with mandatory UPI Autopay

- One plan: period `daily`, interval `15`, amount `1000` paise (fallback: weekly × 2 if 15-day interval is not allowed).
- Pro checkout always creates a subscription; no one-time payment option.
- 7-day trial via `start_at`; mandate authorised on day 1.
- Webhooks (signature-verified) drive access:

Event	Action
subscription.activated	plan = pro
subscription.charged	extend current_period_end by 15 days; log payment
subscription.pending	Telegram + email reminder to keep balance
subscription.halted	plan = free
subscription.cancelled	keep Pro until current_period_end, then free

The bank sends the RBI-required pre-debit notice 24 hours before each charge. Students can cancel the mandate from their UPI app at any time.

Security and compliance (DPDP Act 2023)

- Clear consent at signup for processing resume data; privacy policy and terms pages.
- "Delete my account" removes profile, resumes and outreach logs.
- Resumes in a private Supabase bucket with signed URL.
- Secrets only in environment variables; webhook signat sures verified.
- Never sell or share HR contact lists; honour do-not-contact flags.
- Store Gmail OAuth tokens encrypted; request only the send scope.

4. Step-by-step execution plan

Build in 4 weeks, launch to your own college at the end of week 4, and reach 1,000 signups by day 60.

Day 0: Accounts to open (all free)

- [ ] GitHub repo + GitHub Actions
- [ ] Vercel account linked to the repo
- [ ] Supabase project
- [ ] Google AI Studio key (Gemini) and Groq key
- [ ] RapidAPI (JSearch) and Adzuna developer keys
- [ ] Telegram bot via @BotFather
- [ ] Brevo or Resend account + verified sender domain
- [ ] Razorpay account (PAN + bank); request Subscriptions + UPI Autopay activation now, since approval takes days
- [ ] Hunter.io and Apollo free accounts
- [ ] Domain name (optional; ~₹100–800/year, or use the free vercel.app subdomain)

Week 1: Foundation and resume

1. Create Next.js app with Tailwind; deploy to Vercel.
2. Set up Supabase Auth (Google + email OTP) and all tables from the data model with row-level security.
3. Build onboarding: upload resume → Gemini parse → editable profile form.
4. Build preferences page: roles, locations, job type, alert frequency, channels.
5. Write privacy policy, terms, and consent checkbox.

Done when: a new user can sign up, upload a resume and save a profile.

Week 2: Jobs engine

1. Write fetchers for JSearch, Adzuna, and 20–50 company boards on Greenhouse/Lever.
2. Add dedupe hash and the AI enrich step (skills, fresher flag, contact email).
3. Implement the matching score and save matches.
4. Schedule fetch + match daily on GitHub Actions.
5. Build the dashboard: top matches with score, reasons, Apply and Save buttons.

Done when: each test user sees 10+ relevant new jobs every morning.

Week 3: Alerts and resume tailoring

1. Telegram bot: deep-link connect, digest messages with inline buttons.
2. Email digest template via Brevo/Resend.
3. Alert scheduler for daily / weekly / monthly with plan limits.
4. Resume tailoring: prompt, before/after diff view, PDF export.
5. Usage counters to enforce free vs Pro limits.

Done when: 10 friends receive alerts on Telegram and download a tailored PDF.

Week 4: HR outreach, payments, launch

1. HR email finder chain with caching and do-not-contact.
2. AI email draft + LinkedIn note; copy + open Gmail compose (Gmail API send later).
3. Application tracker and follow-up reminders.
4. Razorpay subscription checkout with trial, webhooks, billing page with next debit date and cancel info.
5. Test end to end with a real ₹10 mandate.
6. Launch to 50–100 students from your college.

Done when: a stranger can sign up, set autopay and get value without your help.

Days 31–60: Growth

- Recruit 1 ambassador per college in Kolhapur, Pune and Sangli; reward with free Pro.
- Run a public Telegram channel with daily top fresher jobs and a bot link.
- Post before/after resume transformations on Instagram and LinkedIn.
- Offer placement cells free batch access in exchange for promotion.
- Collect feedback weekly; fix the top 3 complaints each week.

Month 3+ roadmap

- Gmail API sending after Google OAuth verification
- Skill-gap report and free learning links
- ATS score checker and interview-prep questions per job
- WhatsApp alerts once revenue covers per-message costs

- Prepaid packs (₹30 / 2 months, ₹99 / 6 months) alongside autopay
- College/B2B plan for placement cells

5. Risks and mitigations

The biggest risks are free-tier limits, LinkedIn compliance, and payment friction on tiny UPI mandates; each has a planned fallback.

Risk	Impact	Mitigation
Scraping LinkedIn or auto-messaging gets accounts banned or legal notices	High	Use aggregator APIs for LinkedIn-sourced jobs; LinkedIn notes are copy-and-send by the student
AI free-tier rate limits	High	AI only for parse/enrich/tailor; batch + cache; Groq fallback
Job API quota exhausted	Medium	3+ sources; company boards are unlimited and free
Razorpay subscription approval delayed	Medium	Apply on day 0; run month 1 free if needed
Autopay setup friction on ₹10	Medium	7-day trial, clear "cancel anytime" copy, prepaid packs later
Failed autopay debits	Medium	Razorpay retries + Telegram reminders; downgrade on halt
Student Gmail flagged as spam	Medium	15 emails/day cap, personalized drafts, verified addresses only
Resume tailoring invents skills	High (trust)	Rephrase-only prompt, diff review, user must approve
Personal data misuse (DPDP Act)	High	Consent, delete-my-data, private storage, no list selling
Google OAuth verification takes weeks	Low	Copy + open Gmail compose until approved

LinkedIn compliance rules

- No scraping of LinkedIn pages, profiles or jobs by your servers.
- No automated connection requests or messages.
- LinkedIn-sourced listings only through licensed aggregators such as JSearch.
- The "Apply" button opens the original listing; the student applies themselves.
-

6. AI news articles (GeeksFinder Blog)

GeeksFinder publishes 1–3 original articles a day on hiring and tech-career news, written by Claude from several sources, checked automatically, approved by a human, then shared to the Telegram channel to pull in new users.

Ground rules (copyright, SEO, trust)

- **Never rewrite a single article.** Paraphrasing one source is still copying its work and can draw takedowns. Every article combines facts from at least 2–3 sources and adds GeeksFinder's own angle.
- **Facts are free, wording is not.** Extract facts (who, what, numbers, dates), then write from the facts, never from the source text.
- **Always credit and link sources** at the end of each article.
- **Natural writing, not detector-dodging.** Articles must read like a good human editor wrote them because they are genuinely useful and well written, not because they are tuned to fool AI detectors. Google's spam policies target mass-produced, low-value content however it was made; quality and originality are what rank.
- **A human approves every article** before it goes live, and the byline says "GeeksFinder Desk, reviewed by <name>".

Topics that fit the audience

Company hiring drives and off-campus openings, fresher salary trends, layoffs and what they mean for freshers, in-demand skills, internship programs, exam and certification updates, and interview-process changes at big recruiters.

Pipeline

1. **Fetch (every 6 h, GitHub Actions):** RSS feeds of Indian tech and business outlets, Google News RSS for queries like "fresher hiring", company newsroom pages, and a free news API (GNews free tier; NewsAPI's free plan is for development only, so check terms before production).
2. **Filter and cluster:** drop duplicates; group stories about the same event; keep clusters with 2+ sources and a relevance score above threshold (cheap model: Gemini Flash).
3. **Extract facts:** for each source, pull a JSON list of facts with the source URL attached to each fact. Fetched page text is treated as untrusted data, never as instructions.
4. **Write (Claude):** Claude writes from the fact list only, using the GeeksFinder style guide below, with a required section "What this means for freshers" and 3 matching live jobs from our database.
5. **Automated checks:**
   - Overlap check: no run of 8+ consecutive words shared with any source.
   - Fact check: every number and name in the draft must exist in the fact list.
   - Banned-phrase and length check (600–1,000 words).
6. **Human review queue (admin page):** approve, edit or reject; nothing auto-publishes.
7. **Publish:** blog page with sources list, Article schema markup, OG image, sitemap update; post a short summary and link to the GeeksFinder Telegram channel.

Writing style for natural, human-quality articles

- Lead with the news in one sentence, then why it matters to a final-year student.
- Concrete details: company, role, city, CTC range, dates.
- Vary sentence length; short paragraphs; plain Indian English.
- Never use filler like "In today's fast-paced world", "delve", "game-changer", "unlock", "landscape".
- No invented quotes. Quote only what a source actually reported, attributed.
- End with an action: "See 12 matching fresher jobs" linking to signup.

Model and cost

Claude via the Anthropic API is paid per token (no free tier), so use the smallest Claude model that writes well enough (e.g. Claude Haiku) for drafts, and cap output to 3 articles a day. Use free Gemini for the cheap steps (filtering, fact extraction). Check Anthropic's current pricing before launch.

Data model additions

Table	Key columns
news_sources	id, name, type (rss/api/page), url, active
news_items	id, source_id, url, title, published_at, text_hash, cluster_id
article_drafts	id, cluster_id, facts (jsonb), body_md, overlap_score, factcheck_passed, status (draft/approved/rejected/published), reviewer_id
articles	id, slug, title, body_html (sanitised), sources (jsonb), published_at, telegram_posted

7. Security checklist

GeeksFinder holds resumes, phone numbers, Gmail tokens and payment mandates, so all 19 standard SaaS holes plus 14 risks specific to this product must be closed before launch.

The 19 standard holes, mapped to GeeksFinder

Hole	Risk in GeeksFinder	Fix	How to verify
Row level security	One student reads another's resume or matches	Supabase RLS on every table: `user_id = auth.uid()`	Query another user's row with a second test account; must return nothing
Auth on every API route	Anyone calls `/api/tailor` and burns AI quota	Check session in every route handler and server action	Call each route without a cookie; expect 401
Never trust the frontend	User sets `plan = pro` from the browser	Plan, quotas and prices read only from the database, set only by webhooks	Edit request body in DevTools; server must ignore it
Rate limiting	Spam signups, OTP floods, AI cost abuse	Upstash Ratelimit (free tier) per IP and per user on auth, OTP, AI and email routes	Fire 50 requests in a loop; expect 429

Hole	Risk in GeeksFinder	Fix	How to verify
Sanitize user input	XSS via profile fields or article HTML	Validate with zod; render React text (no `dangerouslySetInnerHTML`); sanitise Markdown with DOMPurify	Save `<img src=x onerror=alert(1)>` as a skill; nothing executes
Parameterized queries	SQL injection	Supabase client / query builder only; no string-built SQL	Search for template-literal SQL in code review
Lock down CORS	Other sites call your API	Allow only your domain; no `*` with credentials	`curl` with a fake Origin header; no allow header returned
HTTPS everywhere	Tokens sniffed on public Wi-Fi	Vercel HTTPS + HSTS header	SSL Labs test grade A
Stop IDOR bugs	`/api/resume/123` returns someone else's PDF	Every lookup filters by owner; use UUIDs, not counting IDs	Change IDs in URLs with a second account
Debug mode off in prod	Stack traces leak keys and schema	Generic error messages; details only in server logs/Sentry	Trigger an error in prod; response shows no stack
CAPTCHA on public forms	Bot signups drain free AI	Cloudflare Turnstile (free) on signup, OTP and contact forms	Submit form via script without token; rejected
Log admin actions	Untraceable changes to users or articles	`audit_log` table: who, what, when, before/after	Approve an article; audit row exists
Private storage buckets	Resumes publicly downloadable	Private bucket + short-lived signed URLs (≤ 10 min)	Open a resume URL logged out after expiry; 403
Hide admin pages properly	Anyone opens `/admin`	Server-side role check (`role = admin` in DB), not just a hidden link	Visit `/admin` as a normal user; 404 or 403

Hole	Risk in GeeksFinder	Fix	How to verify
Update vulnerable packages	Known CVEs in dependencies	Dependabot + `npm audit` in CI weekly	CI fails on high-severity advisories
Check file upload types	Malicious files disguised as PDF	Check magic bytes, allow only PDF, max 2 MB, parse in a sandboxed function	Upload a renamed `.exe`; rejected
Claude security review	Missed logic bugs	Run a Claude Code security review on every PR touching auth, payments or uploads	Review notes attached to PR
Security headers	Clickjacking, XSS, sniffing	CSP, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy` in `next.config`	securityheaders.com grade A
Verify webhook signatures	Fake "payment successful" calls unlock Pro	Verify Razorpay HMAC signature with webhook secret; reject otherwise	Send an unsigned webhook; 400 and no plan change

14 extra risks specific to GeeksFinder

Risk	Where	Fix
Prompt injection	Resumes, job descriptions and news pages can contain text like "ignore instructions"	Treat all fetched/uploaded text as data in delimited blocks; JSON schema outputs; never let AI output trigger actions (sending email, changing plan) without user click
SSRF	Careers-page scraper and news fetcher fetch arbitrary URLs	Allow only http/https on public IPs; block localhost, 169.254.x.x and private ranges; timeouts and size limits

Risk	Where	Fix
Webhook replay and duplicates	Razorpay retries the same event	Store event IDs; process each once (idempotency)
Telegram webhook spoofing	Fake updates link a chat to another account	Set Telegram `secret_token` and check the header; one-time, expiring link codes
Cron endpoint abuse	Public `/api/cron/*` triggers mass alerts	Require a `CRON_SECRET` bearer header
Leaked service-role key	Full DB access if exposed in client bundle	Service key only in server env vars, never `NEXT_PUBLIC_*`; GitHub secret scanning on
Gmail OAuth token theft	Attacker sends mail as students	Encrypt tokens at rest; request send-only scope; let users revoke
Email header injection	Newlines in subject/recipient fields	Validate emails with zod; use the Gmail API's structured fields
AI cost abuse (denial of wallet)	Scripts hammer tailoring or article generation	Per-user quotas in DB, global daily spend cap, alert at 80%
Account enumeration	Login reveals which emails exist	Same message for known and unknown emails
Open redirect	`?next=` param sends users to phishing pages	Allow only relative internal paths
PII in logs	Phone numbers and resume text in logs	Redact before logging; short log retention
Generated article XSS	AI output contains script or iframes	Convert Markdown to HTML server-side and sanitise; strict CSP
Data deletion and backups (DPDP Act)	Can't honour deletion or recover from a bad migration	Working "delete my data" flow; weekly DB backups; test restore once

Prompt for Claude Code

```
Act as a senior application security engineer. Audit this repo against
section 7 of the GeeksFinder plan. For each item, report pass or fail with
file and line, fix every failure, add tests for auth, IDOR, webhook signature
and rate limits, and list anything you could not verify.
```

8. SEO plan

GeeksFinder wins search with three page types that add something new (city × role job hubs, free resume tools, and reviewed career news), on a technically clean Next.js site; thin auto-generated pages are the main risk to avoid.

Target keywords (start here, validate in Google Search Console)

Cluster	Example queries	Page type
Fresher jobs by city and role	"fresher React jobs Pune", "Java developer fresher jobs Bangalore"	Job hub pages
Batch-specific	"off campus drive 2027 batch", "2026 batch fresher hiring"	Job hubs + blog
Resume help	"resume for freshers", "ATS resume checker free", "resume for internship"	Free tools + guides
Outreach	"how to email HR for job", "cold email to recruiter for fresher"	Free tool + guide
News	"<company> fresher hiring 2026", "<company> off campus drive"	Blog articles

Check real search volume and difficulty with free tools (Google Keyword Planner, Search Console data after launch) before building many pages.

Page types

1. **Job hubs** (`/jobs/react-developer-fresher-pune`): live matched jobs plus unique data GeeksFinder owns: number of openings this week, common skills asked, salary range seen, top hiring companies. Only create a hub when it has at least 10 live jobs;

otherwise `noindex`. This avoids thin "programmatic" pages that Google treats as spam.

2. **Free tools** (link magnets, no login): ATS resume checker, "tailor my resume" demo for one job, HR email generator. Each tool ends with "Get matches like this every morning".
3. **Blog**: reviewed news articles (section 6) plus evergreen guides: "How to email HR as a fresher", "Resume format for freshers in India", "How off-campus hiring works".
4. **Core pages**: landing, pricing, about, editorial policy, contact, privacy, terms, refund policy.

Technical SEO checklist

Item	How in Next.js
Unique title + meta description per page	`generateMetadata`; titles under ~60 characters, e.g. "React Fresher Jobs in Pune (Updated Daily) – GeeksFinder"
Canonical URLs	`alternates.canonical`; one URL per page, lowercase, hyphenated
Sitemaps	`app/sitemap.ts` split into jobs, hubs, blog; update on publish
robots.txt	`app/robots.ts`; block `/dashboard`, `/admin`, `/api`, auth pages
noindex private and thin pages	`robots: { index: false }` on app pages, empty hubs, search results
Structured data	`Organization` + `WebSite` on home; `Article` / `NewsArticle` on blog; `FAQPage` content on pricing; `BreadcrumbList` on hubs
JobPosting schema	Only on public job pages that show the full description and a direct apply link; include `validThrough`; remove or update when a job expires (follow Google's job posting guidelines)
Expired jobs	Mark expired, keep page with "similar jobs" or return 404/410; never leave stale jobs indexed

Item	How in Next.js
Open Graph images	`next/og` auto-generated cards for hubs and articles (WhatsApp/Telegram previews matter for this audience)
Core Web Vitals	Budgets from the design guide: LCP < 2.5 s, INP < 200 ms, CLS < 0.1 on mobile
Rendering	Server-render all public pages; no content that only appears after client JS
Internal links	Article → hubs → jobs; breadcrumbs; "related hubs" block on each hub
Image SEO	`next/image`, descriptive alt text, WebP/AVIF
Fast indexing	Google Search Console + Bing Webmaster Tools; submit sitemaps; IndexNow for Bing on new articles
Duplicate job text	Job descriptions copied from sources are duplicate content: lead each job page with GeeksFinder's own summary (skills, fit, fresher-friendly flag) and keep `noindex` on single job pages until they add unique value

Trust signals (E-E-A-T)

- About page with the real founder story and contact details.
- Editorial policy page: how articles are written with AI, checked and human-reviewed.
- Reviewer name and author page on every article.
- Sources linked on every article; corrections note when something changes.

AI search visibility

Clear question-and-answer sections on guides and FAQ ("How much does GeeksFinder cost?" → "₹10 every 15 days"), consistent brand facts across the site, Telegram channel and social profiles, so AI assistants and Google's AI answers describe GeeksFinder correctly.

Off-page growth

- College placement-cell and student-club pages linking to free tools.
- Guest guides for student communities and college blogs.
- Share every article to the GeeksFinder Telegram channel, LinkedIn and Instagram.
- Never buy links or join link networks.

Measurement

Metric	Tool	Target by month 3
Indexed pages	Search Console	90%+ of submitted sitemap URLs
Organic clicks	Search Console	3,000 / month
Signups from organic	Plausible or Umami (free/self-hosted) or GA4	20% of all signups
Core Web Vitals	Search Console + PageSpeed Insights	All URLs "Good"

Prompt for Claude Code

```
Act as a senior technical SEO engineer. Audit GeeksFinder against section 8 of the plan. Implement generateMetadata, canonical URLs, sitemap.ts, robots.ts, structured data and next/og images. Make sure private and thin pages are noindex. Run Lighthouse SEO and PageSpeed on mobile and fix every issue. Report what you changed.
```